

Multirate Sampling with Stein Variational Gradient Descent (Working Title)

Arash Sarshar

California State University, Long Beach

Abstract

We study multirate variants of Stein variational gradient descent (SVGD) that separate the attractive drift and repulsive kernel terms so they can be integrated on different time scales. **[TODO: quantitative takeaway for the first sentence]** We evaluate on 50D Gaussian, 2D target, UCI logistic regression, mixture-of-Gaussians, and Bayesian neural network benchmarks. The results highlight when multirate SVGD improves stability or efficiency over vanilla SVGD while remaining competitive with SGLD and SGHMC.

Keywords: Stein variational gradient descent, multirate integration, particle methods, kernel Stein discrepancy

1. Introduction

Modern Bayesian inference often requires efficient sampling from complex, high-dimensional, and multimodal posterior distributions. As these problems grow in scale and complexity, traditional sampling methods face significant computational and statistical challenges. The advent of automatic differentiation has enabled the widespread use of gradient-based samplers, making methods such as stochastic gradient Langevin dynamics (SGLD) and stochastic gradient Hamiltonian Monte Carlo (SGHMC) practical and popular choices for large-scale applications [1, 2]. While these approaches advance a single Markov chain using noisy gradients, an alternative paradigm is offered by interacting particle methods, which evolve a population of particles whose empirical distribution approximates the target posterior. This population-based perspective opens new avenues for improving sampling efficiency and robustness, especially in challenging inference settings.

A useful unifying perspective is to view many such methods as discretizations

of a particle flow (possibly with a stochastic component) that transports an initial set of particles toward the target distribution. In this work, we focus on Stein variational gradient descent (SVGD) [3] as a representative interacting particle method. SVGD constructs a velocity field for the particles based on the target density and a reproducing kernel Hilbert space (RKHS) structure, and updates the particles by discretizing this flow. The explicit form of the SVGD update and its velocity field are given in eqs. (1) and (2) in Section 2. The velocity field naturally separates into an attractive drift (moving particles toward high-density regions) and a repulsive interaction (spreading particles to maintain diversity and support coverage of multiple modes).

Despite its conceptual simplicity, SVGD presents a numerical challenge that is central to this paper. The drift and repulsion components can impose different stability constraints on the discretization: when particles crowd, repulsion can become stiff and destabilize updates, while on anisotropic targets the drift can require finer resolution to faithfully follow the geometry of $\log p$. A single step size must therefore balance competing requirements and can either over-damp repulsion (leading to loss of diversity and mode coverage) or under-resolve drift (leading to slow progress). Moreover, repulsion typically requires $O(N^2)$ kernel evaluation per step, so increasing stability by oversampling repulsion can become computationally expensive.

A common approach to solving systems with significantly different components is to use partitioned methods such as implicit-explicit (IMEX) [4, 5] and multirate integration methods [6]. In the context of ordinary differential equations (ODEs), multirate methods exploit a natural splitting of the dynamics into fast and slow components, allowing each to be integrated with a step size appropriate to its scale [7, 8, 9]. For example, in a split ODE of the form $\dot{x} = \phi_{\text{fast}}(x) + \phi_{\text{slow}}(x)$, multirate schemes apply more frequent, smaller steps to the stiff or rapidly varying part, while updating the slow part less often. This approach improves both stability and efficiency in multi-scale dynamical systems, and has been successfully applied in a variety of scientific computing contexts.

The structure of the SVGD update naturally separates into two forces: an attractive drift and a repulsive interaction. These forces often operate on different time scales and can impose distinct numerical challenges. Motivated by advances in multirate time-integration for multi-scale dynamical

systems—including MrGARK, IMEX, and related splitting schemes [7, 8, 9, 5, 10]—we propose to integrate the drift and repulsion terms on separate time scales. This leads to a family of multirate SVGD variants, including symmetric splitting, fixed repulsion substepping, and adaptive error-controlled drift substepping.

To assess the effectiveness of these methods, we conduct a comprehensive empirical evaluation. We compare our multirate SVGD variants with established approaches from the literature across a range of benchmark problems, using a variety of task-appropriate metrics. Results highlight the strengths and limitations of each method under diverse geometric and statistical challenges.

1.1. Contributions

Our main contributions are:

- Multirate SVGD formulations that explicitly separate drift and repulsion and support symmetric splitting and fixed multirate schedules.
- An adaptive error-controlled multirate SVGD variant that selects drift substeps based on a local error estimator while keeping repulsion stable.
- A benchmark suite spanning 50D, 2D, UCI, mixture, and BNN targets, with diagnostics (KSD, ESS, mode coverage) and KSD-based early stopping, and with gradient/kernel evaluation accounting for efficiency comparisons.

Organization.. Section 2 presents the SVGD variants and the multirate construction. Section 3 describes the benchmarks and experimental protocol and reports results. Section 4 summarizes findings and discusses directions for future work.

2. Method

2.1. Background: SVGD as a particle flow

Let $p(x)$ denote a target density on $\mathbb{R}^d$ and let $\{x_i^t\}_{i=1}^N$ be a set of particles at iteration t . Stein variational gradient descent (SVGD) evolves particles by following a deterministic interacting-particle flow whose velocity field is

constrained to a reproducing kernel Hilbert space (RKHS) [3]. A first-order discretization of this flow is the update

$$x_i^{t+1} = x_i^t + h \phi(x_i^t), \quad (1)$$

where $h > 0$ is a step size and the SVGD velocity field is

$$\phi(x) = \frac{1}{N} \sum_{j=1}^N \left[k(x_j, x) \nabla_{x_j} \log p(x_j) + \nabla_{x_j} k(x_j, x) \right]. \quad (2)$$

The first term in eq. (2) is an *attractive* drift that pushes particles toward high-density regions via the score $\nabla \log p$. The second term is a *repulsive* interaction that discourages particle collapse.

Throughout this paper, we limit our focus to an RBF kernel

$$k(x, y) = \exp \left(- \frac{\|x - y\|_2^2}{2 h_k} \right), \quad (3)$$

where the bandwidth h_k .¹

2.2. Drift–repulsion splitting and multirate integration

The central numerical observation motivating this work is that the drift and repulsion components of eq. (2) can impose different stability constraints and can naturally evolve on different time scales. We therefore write the SVGD flow as a split system

$$\dot{x} = f_{\text{rep}}(x) + f_{\text{drift}}(x), \quad (4)$$

where, for particle i ,

$$f_{\text{drift}}(x_i) := \frac{1}{N} \sum_{j=1}^N k(x_j, x_i) \nabla_{x_j} \log p(x_j), \quad (5)$$

$$f_{\text{rep}}(x_i) := \frac{1}{N} \sum_{j=1}^N \nabla_{x_j} k(x_j, x_i). \quad (6)$$

We next define multirate discretizations that apply different substepping strategies to f_{drift} and f_{rep} while keeping a common macro-step size h .

¹The bandwidth h_k is set by the median heuristic (the median pairwise squared distance among particles) and scaled by a factor $\alpha > 0$ to modulate the strength of repulsion.

Strang-split SVGD. We can define the forward Euler steps that advances the split system forward by a stepsize τ as:

$$\Psi_{\text{rep}}^\tau(x) = x + \tau f_{\text{rep}}(x), \quad \Psi_{\text{drift}}^\tau(x) = x + \tau f_{\text{drift}}(x).$$

Strang splitting [11] is a classical second order method taht construct a macro-step of size h by composing such substeps. In our setting, one macro step takes the form

$$x^{t+1} = \Psi_{\text{rep}}^{h/2} \circ \Psi_{\text{drift}}^h \circ \Psi_{\text{rep}}^{h/2}(x^t), \quad (7)$$

which alternates half a repulsion step, a full drift step, and another half repulsion step. In practice the half-steps can in fact be implemented as $\frac{h}{2m}$ smaller substeps for stability.

Fixed multirate SVGD (MR-SVG). Fixed multirate integrators provide a principled way to allocate different step sizes to fast and slow components of a split system [6, 7, 8]. In our SVGD setting, however, higher-order multirate schemes are often not cost-effective: their additional stages translate into multiple evaluations of the drift and repulsion fields per macro step, and each such evaluation is expensive due to gradient computations and, in particular, $O(N^2)$ kernel interactions. Moreover, practical accuracy is also limited by finite-particle effects and by choices such as bandwidth selection and early stopping, which can mute the benefits of increasing time-integration order at a fixed computational budget. We therefore specialize to a first-order multirate construction (Euler update maps) that preserves a macro step of size h for the drift while resolving the repulsion with m substeps:

$$x^{t+1} = \Psi_{\text{drift}}^h \circ \left(\Psi_{\text{rep}}^{h/m} \right)^m (x^t). \quad (8)$$

This retains a coarse drift step while stabilizing the repulsive interaction when particles crowd, at the cost of additional kernel evaluations. The corresponding implementation-level procedure is summarized in algorithm 1.

Adaptive multirate SVGD (Adapt-MR-SVG). Although fixed- m schedules are simple and robust, they impose a uniform drift resolution along the entire trajectory. As a result, they may spend unnecessary computational effort in benign regions while providing insufficient resolution in locally stiff regimes. To address this imbalance, we choose the number of drift microsteps m adaptively at each macro step, while retaining one repulsion update per macro step. This follows the standard local-error-control perspective used in ODE

Algorithm 1 Fixed multirate SVGD (MR-SVGD)

Require: Particles $\{x_i\}_{i=1}^N$, step size h , repulsion substeps m , kernel k

```
1:  $x \leftarrow \{x_i\}$ 
2: for  $s = 1$  to  $m$  do
3:   compute repulsion  $f_{\text{rep}}(x)$  using eq. (2)
4:    $x \leftarrow x + (h/m) f_{\text{rep}}(x)$ 
5: end for
6: compute drift  $f_{\text{drift}}(x)$  using eq. (2)
7:  $x \leftarrow x + h f_{\text{drift}}(x)$ 
8: return  $x$ 
```

integrators, where local error estimates are used to adjust resolution [12]. After the repulsion update, denote the state by $x = \Psi_{\text{rep}}^h(x^t)$. We estimate the local drift-discretization error by comparing one drift step of size h with two consecutive drift half-steps of size $h/2$,

$$x_{\text{full}} = x + h f_{\text{drift}}(x), \quad (9)$$

$$x_{\text{half}} = x + \frac{h}{2} f_{\text{drift}}(x), \quad (10)$$

$$\tilde{x}_{\text{full}} = x_{\text{half}} + \frac{h}{2} f_{\text{drift}}(x_{\text{half}}). \quad (11)$$

We then form the relative discrepancy

$$\rho = \frac{\|\tilde{x}_{\text{full}} - x_{\text{full}}\|_2}{\|x\|_2 + \epsilon}. \quad (12)$$

which serves as a dimensionless local error indicator for the drift update. For a first-order drift integrator, the step-doubling discrepancy behaves locally as

$$\rho(h) = Ch^2 + \mathcal{O}(h^3).$$

If the drift step is replaced by m microsteps of size h/m , then

$$\rho_m \approx C(h/m)^2 \approx \rho/m^2.$$

Imposing the target local accuracy $\rho_m \approx \varepsilon$ gives $m \approx \sqrt{\rho/\varepsilon}$. We therefore use the clipped integer controller

$$m \leftarrow \text{clip}\left(\left\lceil \sqrt{\rho/\varepsilon} \right\rceil, m_{\min}, m_{\max}\right), \quad (13)$$

Algorithm 2 Adaptive multirate SVGD (Adapt-MR-SVGD)

Require: Particles $\{x_i\}_{i=1}^N$, step size h , bounds $m_{\min}, m_{\max}$, tolerance ε

- 1: $x \leftarrow \{x_i\}$
- 2: compute repulsion $f_{\text{rep}}(x)$ using eq. (6)
- 3: $x \leftarrow x + h f_{\text{rep}}(x)$
- 4: compute drift $f_{\text{drift}}(x)$ using eq. (5)
- 5: $x_{\text{full}} \leftarrow x + h f_{\text{drift}}(x)$
- 6: $x_{\text{half}} \leftarrow x + (h/2) f_{\text{drift}}(x)$
- 7: compute drift $f_{\text{drift}}(x_{\text{half}})$
- 8: $\tilde{x}_{\text{full}} \leftarrow x_{\text{half}} + (h/2) f_{\text{drift}}(x_{\text{half}})$
- 9: estimate relative error $\rho = \|\tilde{x}_{\text{full}} - x_{\text{full}}\| / (\|x\| + \epsilon)$
- 10: $m \leftarrow \text{clip}(\lceil \sqrt{\rho/\varepsilon} \rceil, m_{\min}, m_{\max})$
- 11: **if** $m \leq 2$ **then**
- 12: $x \leftarrow \tilde{x}_{\text{full}}$ $\triangleright$ reuse step-doubling path
- 13: **else**
- 14: **for** $s = 1$ to m **do**
- 15: compute drift $f_{\text{drift}}(x)$
- 16: $x \leftarrow x + (h/m) f_{\text{drift}}(x)$
- 17: **end for**
- 18: **end if**
- 19: **return** x

which increases m when $\rho > \varepsilon$ and decreases it when $\rho \leq \varepsilon$, while enforcing the robustness bounds $m_{\min} \leq m \leq m_{\max}$. The macro-step update is then

$$x^{t+1} = \left(\Psi_{\text{drift}}^{h/m} \right)^m \circ \Psi_{\text{rep}}^h(x^t), \quad m = m(x^t, h). \quad (14)$$

Here $\varepsilon > 0$ is a user-set tolerance and $\epsilon > 0$ is a small constant to avoid division by zero. This strategy increases drift resolution only when the local error indicator suggests stiffness or rapid variation in the drift component. See algorithm 2 for complete implementation.

Remark 1. In implementation, if $m \leq 2$ we reuse the already computed step-doubling trajectory to avoid extra drift evaluations.

Whitening and numerical guards. For high-dimensional Gaussian targets, we optionally transform particles using a whitening matrix L^{-1} derived from the target covariance. We also clip gradients and clamp extreme values to avoid

non-finite updates; these guards do not change the fixed point but improve robustness.

3. Experiments and Results

We evaluate multirate SVGD variants on five benchmark groups: a 50D Gaussian target, a suite of 2D synthetic targets, UCI logistic regression, a 2D Gaussian mixture (mix8), and a one-hidden-layer Bayesian neural network (BNN). The comparison focuses on both statistical quality and computational cost.

3.1. Experimental protocol

Across benchmarks, we compare the same method family whenever applicable: vanilla SVGD, Strang-split SVGD, fixed multirate SVGD (MR-SVGD), adaptive multirate SVGD (Adapt-MR-SVGD), and the single-chain baselines SGLD and an SGHMC-style method. The SVGD-family updates are those defined in section 2, namely eqs. (1), (7), (8) and (14).

Remark 2. For the single-chain baselines, we use the standard stochastic-gradient update

$$x_{t+1} = x_t + \eta \nabla \log p(x_t) + \sqrt{2\eta} \xi_t, \quad \xi_t \sim \mathcal{N}(0, I), \quad (15)$$

for SGLD [1], and

$$v_{t+1} = \beta v_t + \eta \nabla \log p(x_t) + \sqrt{2\alpha\eta} \xi_t, \quad x_{t+1} = x_t + v_{t+1}, \quad (16)$$

for an SGHMC-style baseline [2]. Here η is the step size, α is a friction/noise parameter, and $\beta \in (0, 1)$ is momentum decay.

For particle methods, we use $N = 128$ particles; for single-chain methods, we compute metrics from a rolling window of recent iterates to obtain comparable finite-sample summaries. Alongside wall-clock time, we report gradient- and kernel-evaluation counts as hardware-agnostic measures of computational cost.

All benchmarks use checkpointed evaluation at interval `save_every` and an early-stopping rule triggered by persistent degradation of a designated monitor metric. We use KSD as the monitor for 50D Gaussian, 2D synthetic targets, Mixture2D, and BNN (`COMPUTE_KSD=True` in the BNN setup), whereas UCI logistic regression uses NLL as the monitor because its default configuration sets `COMPUTE_KSD=False`.

Missing figure: ../figures/50d/pareto_mu_cov_final_mean_err.png

Figure 1: 50D Gaussian: final mean $\pm$ std Pareto plot in moment-error space with marker size encoding ESS (left) and wall time (right).

Missing figure: ../figures/50d/summary_metric_panels.png

Figure 2: 50D Gaussian final-checkpoint summary across methods. The four panels report $\|\hat{\mu} - \mu\|_2$, $\|\hat{\Sigma} - \Sigma\|_F$, KSD, and ESS, respectively; markers show mean values across seeds and error bars indicate one standard deviation. Lower is better for the first three metrics, whereas higher is better for ESS.

3.2. 50D Gaussian

This benchmark stresses stability in a high-dimensional setting with anisotropic covariance structure. We run 1,000 iterations with checkpointing every 50 steps and a 5-seed ablation. Reported metrics are moment errors ($\|\hat{\mu} - \mu\|_2$ and $\|\hat{\Sigma} - \Sigma\|_F$), ESS, KSD, and mean log probability.

3.3. 2D synthetic targets

We evaluate banana, ring, squiggly, two-moons, and funnel targets with 1,000 iterations and checkpointing every 20 steps. Metrics include mean error, covariance error, ESS, KSD, mean log probability, and grid-based L_1 discrepancy against reference densities. The current draft keeps a placeholder for a consolidated multi-target figure; per-target panels are available under `figures/2d/<target>/`.

3.4. Mixture2D (mix8)

This benchmark emphasizes mode discovery and mass allocation across modes. We run 1,000 iterations (checkpoint every 20) across 5 seeds. Metrics include mode coverage, mode entropy, imbalance, minimum mass per mode, ESS, KSD, mean log probability, and grid L_1 discrepancy. The summary panels directly visualize coverage/entropy-quality tradeoffs together with cost-aware curves.

3.5. UCI logistic regression

We benchmark Bayesian logistic regression on breast_cancer, ionosphere, spambase, and a5a, using a Gaussian prior on weights (as implemented in the benchmark script) and 5 seeds per dataset. We report test accuracy,

Missing figure: ../figures/50d/walltime_overview.png

Figure 3: 50D Gaussian: wall-time overview across key metrics.

Placeholder for 2D summary figure.

Figure 4: 2D targets: placeholder for the main figure.

NLL, ECE, ESS, and mean log probability; KSD is optional and disabled by default in this benchmark configuration. Results are summarized with per-dataset panels in fig. 6 and a cross-dataset winner scorecard (particle methods only) in table 2 [13].

3.6. Bayesian neural network

We evaluate a one-hidden-layer BNN (hidden width 32) on `breast_cancer`, `ionosphere`, and `a5a`, using a Gaussian prior and 5 seeds per dataset. We report test accuracy, NLL, ECE, ESS, KSD, and mean log probability, and summarize results with panels analogous to the logistic-regression benchmark.

4. Conclusions

We presented multirate SVGD variants that split drift and repulsion to improve stability and efficiency on stiff targets. Across synthetic and real-world benchmarks, multirate splitting and adaptive drift substepping consistently improve robustness relative to vanilla SVGD, while remaining competitive with SGLD and SGHMC. The results highlight the importance of separating time scales in particle interactions, especially in high dimensions and multimodal settings.

Future work includes more principled adaptivity for repulsion frequency, approximate kernel interactions for large N , and tighter links to classical multirate stability theory for particle-based Bayesian inference.

References

- [1] M. Welling, Y. W. Teh, Bayesian learning via stochastic gradient langevin dynamics, in: International Conference on Machine Learning, 2011.

Method	Med. L_1	Med. KSD	Med. grad evals	Med. wall (s)	Pareto
SVGD	1.964	604.15	180	0.77	Yes
Strang	1.997	57.17	220	2.44	Yes
MR	1.999	68.27	180	3.11	No
Adapt-MR	1.999	1.94	1004	2.89	Yes

Table 1: 2D targets quality-cost summary across particle methods. Entries are medians across targets of final-checkpoint metrics from current outputs (`metrics/2d/*.csv`). Lower is better for L_1 and KSD. The Pareto flag is computed on the (median KSD, median wall-time) plane. Method labels: Strang = Strang-SVGD, MR = MR-SVGD, Adapt-MR = Adapt-MR-SVGD.

Missing figure: `../figures/mixture2d/mix8/mix8_summary_metric_`panels.png

Figure 5: Mixture2D (mix8) final-checkpoint summary across methods. The four panels report mode coverage, mode entropy, minimum mass per mode, and KSD; markers show mean values across seeds and error bars indicate one standard deviation. Higher is better for coverage, entropy, and minimum mass per mode, while lower is better for KSD.

- [2] T. Chen, E. Fox, C. Guestrin, Stochastic gradient hamiltonian monte carlo, in: International Conference on Machine Learning, 2014.
- [3] Q. Liu, D. Wang, Stein variational gradient descent: A general purpose bayesian inference algorithm, in: Advances in Neural Information Processing Systems, 2016.
- [4] L. Pareschi, G. Russo, Implicit-explicit Runge-Kutta schemes for stiff systems of differential equations, in: Recent trends in numerical analysis, Nova Science Publishers, Inc., 2000, pp. 269–288.
- [5] A. Sarshar, S. Roberts, A. Sandu, Parallel implicit-explicit general linear methods, Communications on Applied Mathematics and Computation 3 (2021) 649–669. doi:10.1007/s42967-020-00083-5.
- [6] M. Günther, A. Sandu, Multirate generalized additive Runge-Kutta methods, Numerische Mathematik 133 (3) (2016) 497–524. doi:10.1007/s00211-015-0756-z.
URL <http://dx.doi.org/10.1007/s00211-015-0756-z>
- [7] A. Sarshar, S. Roberts, A. Sandu, Design of high-order decoupled multi-

Missing figure: ../figures/uci/breast_cancer/breast_cancer_summary_metric_panels.png

(a) breast_cancer

Missing figure: ../figures/uci/ionosphere/ionosphere_summary_metric_panels.png

(b) ionosphere

Figure 6: UCI logistic regression summary across datasets. Each subpanel reports final-checkpoint mean across seeds with one standard deviation for test accuracy, NLL, ECE, and ESS. Higher is better for accuracy and ESS; lower is better for NLL and ECE. Remaining datasets are shown in the continued panel.

Missing figure: ../figures/uci/spambase/spambase_summary_metric_panels.png

(c) spambase

Missing figure: ../figures/uci/a5a/a5a_summary_metric_panels.png

(d) a5a

Figure 6: UCI logistic regression summary across datasets (continued).

rate gark schemes, SIAM Journal on Scientific Computing 41 (2) (2019) A816–A847. doi:10.1137/18M1182875.

- [8] A. Sandu, M. Günther, S. Roberts, A. Sarshar, Implicit multi-rate gark methods, Journal of Scientific Computing 87 (2021) 1–32. doi:10.1007/s10915-020-01400-z.
- [9] A. Sarshar, S. Roberts, A. Sandu, Alternating directions implicit integration in a general linear method framework, Journal of Computational and Applied Mathematics 387 (2021) 112496. doi:10.1016/j.cam.2019.112496.
- [10] A. Sarshar, S. Roberts, A. Sandu, A fast time-stepping strategy for dynamical systems equipped with a surrogate model, SIAM Journal on Scientific Computing 44 (3) (2022) A1405–A1430. doi:10.1137/20M1386281.
- [11] G. Strang, On the construction and comparison of difference schemes, SIAM Journal on Numerical Analysis 5 (3) (1968) 506–517.
- [12] E. Hairer, S. Norsett, G. Wanner, Solving ordinary differential equations I: Nonstiff problems, no. 8 in Springer Series in Computational Mathe-

Dataset	Acc $\uparrow$ winner	NLL $\downarrow$ winner	ECE $\downarrow$ winner	ESS $\uparrow$ winner
breast_cancer	MR (0.977)	Adapt-MR (0.075)	SVGD (0.594)	MR (124.0)
ionosphere	SVGD (0.891)	Adapt-MR (0.523)	SVGD (0.378)	MR (127.7)
spambase	SVGD (0.884)	Adapt-MR (0.412)	Adapt-MR (0.541)	SVGD/MR (128.0)
a5a	Strang (0.815)	MR (0.510)	Adapt-MR (0.612)	SVGD (123.2)

Table 2: UCI logistic regression winner scorecard (particle methods only: SVGD, Strang-SVG, MR-SVG, Adapt-MR-SVG). Each entry reports the best method and its final-checkpoint mean value across seeds for the corresponding dataset and metric; ties are shown with “/”.

Missing figure: ../figures/bnn/breast_cancer/breast_cancer_summary_metric_panels.png

(a) breast_cancer

Missing figure: ../figures/bnn/ionosphere/ionosphere_summary_metric_panels.png

(b) ionosphere

Figure 7: BNN summary across datasets. Each subpanel reports final-checkpoint mean across seeds with one standard deviation for test accuracy, NLL, ECE, and ESS. Higher is better for accuracy and ESS; lower is better for NLL and ECE. The remaining dataset appears in the continued panel.

matics, Springer-Verlag Berlin Heidelberg, 1993. doi:10.1007/978-3-540-78862-1.

- [13] D. Dua, C. Graff, Uci machine learning repository, <https://archive.ics.uci.edu/ml>, accessed 2025-01-01 (2019).

Missing figure: ../figures/bnn/a5a/a5a_summary_metric_panels.png

(c) a5a

Figure 7: BNN summary across datasets (continued).